

XP System Security Audit

Target: Gamification XP + Streak System

Focus Areas: 1. XP Manipulation 2. Atomic Database Operations 3. Authorization Enforcement 4. XP Formula Abuse 5. Race Conditions

Status: ✗ Not Approved for Production

Executive Summary

The XP system contains several critical architectural risks that could result in:

- Duplicate XP awards
- Duplicate streak increases
- Lost XP updates
- Data corruption
- Progression inconsistencies
- Replay attacks
- XP inflation

The largest issue is that completion processing appears to rely on multiple separate database operations instead of a single atomic transaction.

Even honest users opening multiple tabs or experiencing network retries can corrupt progression data.

Overall Production Readiness: 3/10

CRITICAL ISSUES

CRITICAL #1: Duplicate Goal Completion Race Condition

Risk Level

CRITICAL

What Is Wrong

The completion flow performs:

```
const alreadyCompletedToday =  
  await checkAlreadyCompletedToday(goalId, userId);  
  
if (alreadyCompletedToday) {  
  throw new AlreadyCompletedTodayError(goalId);  
}
```

Followed later by:

```
await insertCompletion(...);
```

Why It's Wrong

This is a classic Time-of-Check vs Time-of-Use race condition.

Example:

```
Request A -> checkAlreadyCompletedToday() = false  
Request B -> checkAlreadyCompletedToday() = false  
  
Request A -> insert completion  
Request B -> insert completion
```

Both requests succeed.

Impact

Users may receive:

- Double XP
- Double streak increases
- Duplicate completion records
- Multiple level-ups

This is the most likely real-world exploit.

Recommended Fix

Enforce uniqueness in PostgreSQL.

Example:

```
CREATE UNIQUE INDEX unique_daily_completion
ON completions(goal_id, user_id, completion_date);
```

Handle duplicate-key violations properly.

Verification

Run 50 concurrent completion requests.

Expected:

```
1 success
49 duplicate completion failures
```

CRITICAL #2: XP Updates Are Not Atomic

Risk Level

CRITICAL

What Is Wrong

Current logic:

```
const newTotalXP = user.total_xp + xpEarned;

await updateUserXP(
  userId,
  newTotalXP,
  levelResult.newLevel
);
```

Why It's Wrong

Concurrent requests:

Current XP:

```
100
```

Request A:

+50 XP

Request B:

+50 XP

Both read:

100

Both calculate:

150

Both save:

150

Expected:

200

Actual:

150

This is a lost update bug.

Impact

Permanent XP corruption.

User progression becomes inaccurate.

Recommended Fix

Never use:

Read → Calculate → Write

Use atomic SQL updates:

```
UPDATE users
SET total_xp = total_xp + :xpEarned
WHERE id = :userId;
```

Or execute through a transactional RPC.

Verification

Run 100 simultaneous XP awards.

Verify:

```
Final XP =
Starting XP + Sum(All XP Awards)
```

exactly.

CRITICAL #3: Completion Processing Is Not Atomic

Risk Level

CRITICAL

What Is Wrong

The implementation appears to perform:

```
Update XP
Insert Completion
Update Streak
Insert Stats
```

as separate operations.

Why It's Wrong

Example:

```
Update XP -> Success  
Insert Completion -> Failure
```

Result:

```
XP awarded  
No completion record exists
```

The system becomes inconsistent.

Impact

- XP without completion records
- Incorrect streaks
- Incorrect stats
- Broken leaderboards

Recommended Fix

Move all operations into a single database transaction.

Example:

```
BEGIN;  
  
UPDATE users ...  
INSERT completion ...  
UPDATE streak ...  
INSERT stats ...  
  
COMMIT;
```

Or implement through a PostgreSQL RPC function.

Verification

Force failures at each step.

Verify:

Either everything succeeds
OR
Nothing changes

CRITICAL #4: Streak Inflation Through Concurrency

Risk Level

CRITICAL

What Is Wrong

Streak calculations rely on:

```
calculateStreak(  
    goal.last_completed_at,  
    goal.streak_count  
);
```

using potentially stale data.

Why It's Wrong

Two requests:

```
Both read streak = 10  
Both calculate streak = 11  
Both write streak = 11
```

or

```
Both award streak bonus for day 11
```

Impact

- Streak inflation
- Duplicate streak bonuses
- Incorrect progression

Recommended Fix

Calculate streaks inside a transaction.

Lock the relevant row while processing.

Verification

Send concurrent completion requests.

Verify streak only increments once.

HIGH ISSUES

HIGH #1: Dangerous User ID Trust Boundary

Risk Level

HIGH

What Is Wrong

The system accepts:

```
{
  goalId,
  userId,
  quality
}
```

and then performs:

```
fetchGoal(goalId, userId);
fetchUser(userId);
```

Why It's Wrong

If `userId` originates from client input:


```
{  
  "userId": "victim-id"  
}
```

an attacker can attempt actions on behalf of another user.

Impact

Potential unauthorized progression manipulation.

Recommended Fix

Never trust client-provided user IDs.

Always derive identity from:

```
auth.uid()
```

or authenticated session context.

Verification

Modify request payload manually.

Attempt:

```
{  
  "userId": "another-user"  
}
```

Expected:

```
Authorization failure
```

HIGH #2: Self-Reported Quality Is Easily Abused

Risk Level

HIGH

What Is Wrong

XP calculations depend on:

quality

which is user-controlled.

Why It's Wrong

Users will always choose:

10/10

for every completion.

Nothing validates quality.

Impact

- XP inflation
- Leaderboard corruption
- Meaningless progression

Recommended Fix

Options:

Option A

Remove quality entirely.

Option B

Fixed XP per completion.

Option C

Review or moderation system.

Option D

Small capped quality bonus.

Example:

Maximum +10% XP

instead of large multipliers.

Verification

Review XP distribution after large-scale usage.

HIGH #3: Unlimited Streak Scaling

Risk Level

HIGH

What Is Wrong

Current formula:

```
streakBonus = streak * STREAK_BONUS_PER_DAY;
```

No maximum cap exists.

Why It's Wrong

Long streaks eventually dominate all XP gains.

Example:

300-day streak

could produce:

1500 bonus XP

depending on configuration.

Impact

Runaway XP inflation.

New users become non-competitive.

Recommended Fix

Cap streak bonuses:

```
bonus = Math.min(streak, 30) * 5;
```

Or use diminishing returns.

Verification

Simulate:

```
1 day  
30 days  
100 days  
365 days
```

Ensure growth remains controlled.

HIGH #4: Replay Attack Vulnerability

Risk Level

HIGH

What Is Wrong

No idempotency mechanism appears to exist.

Why It's Wrong

Users can:

- Retry requests
- Replay requests
- Exploit network retransmissions

Example:

```
POST /completeGoal
```

replayed repeatedly.

Impact

Duplicate rewards.

Duplicate progression.

Recommended Fix

Add:

```
completion_request_id
```

with a unique constraint.

Process each completion request exactly once.

Verification

Replay the same request 100 times.

Expected:

```
First request succeeds  
All others rejected
```

MEDIUM ISSUES

MEDIUM #1: Streak Logic Depends On Hour Differences

Risk Level

MEDIUM

What Is Wrong

Logic similar to:

```
const daysGap = Math.floor(diffHours / 24);
```

Why It's Wrong

Example:

Yesterday: 11:59 PM
Today: 12:01 AM

Actual elapsed time:

2 minutes

Potentially interpreted incorrectly.

Impact

Unexpected streak behavior.

User frustration.

Recommended Fix

Use calendar-day comparisons.

Not elapsed-hour calculations.

Verification

Test around:

- Midnight
- DST transitions
- Time zone changes

MEDIUM #2: Future Timestamps Can Break Progression

Risk Level

MEDIUM

What Is Wrong

Future completion timestamps may produce:

```
daysGap <= 0
```

and invalid state.

Impact

Users may become unable to continue streaks.

Recommended Fix

Clamp future timestamps and repair invalid values.

Verification

Insert future timestamps in staging.

Verify graceful recovery.

MEDIUM #3: Level-Up Logic Assumes XP Integrity

Risk Level

MEDIUM

What Is Wrong

Level calculations trust:

```
totalXP
```

without validation.

Impact

Corrupted XP values may generate invalid levels.

Recommended Fix

Validate:

```
totalXP >= 0  
totalXP <= MAX_XP
```

before calculations.

Verification

Test:

```
Negative XP  
Very large XP  
Corrupted XP
```

LOW ISSUES

LOW #1: XP Formula Is Easy To Optimize

Risk Level

LOW

Users will eventually discover:

```
Always choose quality = 10  
Maintain streak  
Maximize XP
```

Not a security vulnerability, but a game design weakness.

LOW #2: Stats Aggregation Fails Open

Risk Level

LOW

Returning zeroed stats on aggregation failure can hide corruption.

Prefer:


```
Monitoring alert
Error reporting
```

instead.

Direct Answers To Security Questions

1. Can Users Manipulate XP/Streak Without Exploiting?

Answer

Yes.

Methods include:

- Always selecting maximum quality
- Concurrent request abuse
- Replaying completion requests
- Exploiting duplicate completion races

2. Are Database Operations Atomic?

Answer

No.

Current flow appears to be:

```
Update User
Insert Completion
Update Streak
Insert Stats
```

These are separate operations.

Atomicity is not guaranteed.

3. Is User Authorization Properly Enforced?

Answer

Cannot be proven from current implementation.

Security depends on:

```
fetchGoal()  
fetchUser()
```

and whether identity is derived from authenticated sessions.

If user IDs are accepted from clients, authorization may be bypassed.

4. Can The XP Formula Be Gamed Or Inflated?

Answer

Yes.

Primary vectors:

- Self-reported quality
 - Unlimited streak scaling
 - Replay attacks
 - Duplicate completion processing
-

5. Are There Race Conditions?

Answer

Yes — multiple critical race conditions exist.

1. Duplicate completion race
 2. Lost XP update race
 3. Streak update race
 4. Multi-step transaction race
-

Required Fixes Before Production

1. Add unique completion constraints.
 2. Move completion processing into a single transaction.
 3. Use atomic XP increments.
 4. Derive user identity from authenticated sessions only.
 5. Add idempotency protection.
 6. Cap streak bonuses.
 7. Redesign quality scoring.
 8. Lock rows or use transactional streak calculations.
 9. Add concurrency tests.
 10. Add replay attack tests.
-

Final Verdict

Deployment Recommendation: **BLOCK**

The primary risk is not the XP formula itself.

The primary risk is the combination of:

Race Conditions
+
Non-Atomic Updates
+
User-Controlled Reward Inputs

which can eventually corrupt progression data even without malicious users.

Until atomic transactions, concurrency controls, and idempotency protections are implemented, this XP system should not be considered production-ready.